

Penerapan Praktik SQL

Modul Intermediate Programmer

Pemetaan 12 bab modul *Training Programmer — SQL v2* terhadap implementasi nyata di project Inventory & Order Management System, disertai nama file dan nomor baris sebagai bukti.

PROJECT

IOMS (ioms/)

DISUSUN OLEH

Arief Aryudi Syidik

REFERENSI MODUL

SQL v2, Edisi 2.0

TANGGAL

18 September 2026

- ✓ Diterapkan
- Sebagian
- ✗ Belum / Tidak relevan

BAB	TOPIK	STATUS
01	Workload, pages, access paths	Diterapkan
02	Index architecture & design decisions	Diterapkan
03	Aggregate reports	Diterapkan
04	Self-join, hierarchy, pivot	Tidak relevan
05	Subquery & correlated reasoning	Diterapkan
06	EXISTS/IN/ANY/ALL & NULL semantics	Sebagian
07	Transactions, isolation, failure handling	Diterapkan
08	Views, layered views, stored procedures	Belum diterapkan
09	Triggers	Belum diterapkan (by design)

BAB	TOPIK	STATUS
10	OLTP/OLAP, access control, log, profiling	Sebagian
11	Execution plans & evidence-based tuning	Belum diterapkan
12	N+1 refactoring	Diterapkan

01 Workload, Pages, dan Access Paths

Diterapkan

Grain query jelas per kebutuhan (list per halaman, detail per ID), akses lewat kolom yang sudah diindeks, dan jumlah baris yang dibaca dibatasi lebih dulu sebelum ditampilkan.

app/Controller/PurchaseOrderController.php :40–41 · **app/Controller/SalesOrderController.php :40–41**

Pagination `limit` ⇒ `10` + `offset` dihitung dari halaman aktif — jumlah baris yang di-fetch dan dikirim ke browser dibatasi di query, bukan di-slice di PHP setelah ambil semua data.

app/Repository/MySQLPurchaseOrderRepository.php :86–88

`LIMIT {\$limit} OFFSET {\$offset}` diterapkan di level SQL; filter status/supplier/tanggal masuk `WHERE` sebelum `LIMIT`, sehingga access path memakai index (lihat Bab 2) alih-alih scan penuh lalu filter di aplikasi.

Catatan: tidak ada EXPLAIN plan yang didokumentasikan secara formal untuk membuktikan access path aktual dipakai optimizer — lihat Bab 11.

02 Index Architecture & Design Decisions

Diterapkan

Index dipasang pada kolom yang benar-benar dipakai untuk filter, join, dan uniqueness — bukan index spekulatif.

database/schema.sql :60–63

Produk: uq_products_sku (unique, dipakai validasi SKU duplikat), idx_products_category_id (join/filter kategori), idx_products_is_active (filter status aktif).

database/schema.sql :78–81

Composite unique uq_product_warehouse (product_id, warehouse_id) — sekaligus jadi access path untuk lockForUpdate() di ARCH-02 (Bab 7), plus CHECK (quantity >= 0).

database/schema.sql :127–131, 179–183

Index pada status, supplier_id/customer_id, warehouse_id, created_by di kedua tabel order — persis kolom yang dipakai di buildWhere() repository untuk filter list.

database/schema.sql :231–235

stock_ledger: index pada product_id, warehouse_id, movement_type, (reference_type, reference_id), dan created_at — mendukung laporan CSV per rentang tanggal (Bab 3).

03 Aggregate Reports yang Benar dan Transparan

Diterapkan

Semua angka ringkasan (dashboard & laporan CSV) dihitung lewat COUNT/SUM/GROUP BY/HAVING di database, bukan dihardcode atau dihitung ulang di PHP dari data mentah.

app/Repository/MySqlProductStockRepository.php :101–114

`countLowStock()` — subquery ber-GROUP BY `p.id`, `p.reorder_point` dan HAVING `COALESCE(SUM(ps.quantity), 0) < p.reorder_point`, dibungkus `COUNT(*)` di query luar.

```
SELECT COUNT(*) FROM (
  SELECT p.id FROM products p
  LEFT JOIN product_stocks ps ON ps.product_id = p.id
  WHERE p.is_active = 1
  GROUP BY p.id, p.reorder_point
  HAVING COALESCE(SUM(ps.quantity), 0) < p.reorder_point
) t
```

app/Repository/MySqlSalesOrderRepository.php :177–184

app/Repository/MySqlPurchaseOrderRepository.php :109–116

`countsByStatus()` — satu query GROUP BY `status` per tabel menghasilkan rekap semua status sekaligus (lihat juga Bab 12: sebelumnya ini 8 query terpisah).

app/Service/ReportService.php :24–70

Ekspor CSV stock ledger & status order memakai `Repository::search()` yang sama dengan yang dipakai halaman list — logic agregasi konsisten, tidak ada query terpisah yang bisa berbeda hasil dari dashboard.

04 Self-Join, Hierarchy, dan Pivot

Tidak relevan

Domain IOMS (inventori/order) tidak memiliki struktur hierarkis (mis. kategori bertingkat/organisasi atasan-bawahan) maupun kebutuhan laporan pivot/cross-tab, sehingga tidak ada self-join atau PIVOT yang diterapkan.

Tidak dianggap sebagai kekurangan — brief project tidak meminta fitur hierarki/pivot apa pun.

05 Subquery dan Correlated Reasoning

Diterapkan

Filter status stok (low/normal) memakai scalar correlated subquery yang mereferensikan tabel luar per baris.

app/Repository/MySqlProductRepository.php :66–71

Correlated subquery `ps.product_id = p.id` dievaluasi ulang untuk tiap baris `products` pada saat filter `status=low/normal` diterapkan.

```
WHERE (SELECT COALESCE(SUM(ps.quantity), 0)
FROM product_stocks ps
WHERE ps.product_id = p.id) < p.reorder_point
```

06 EXISTS, IN, ANY, ALL, dan NULL Semantics

Sebagian

Penanganan NULL lewat `LEFT JOIN + COALESCE` sudah konsisten diterapkan; operator `EXISTS/IN/ANY/ALL` secara literal belum dipakai karena belum ada kebutuhan query yang pas untuknya.

app/Repository/MySQLProductStockRepository.php :106, 109, 121, 124

`LEFT JOIN product_stocks ps` memastikan produk tanpa baris stok sama sekali tetap ikut agregasi (bukan hilang seperti `INNER JOIN`), dan `COALESCE(SUM(ps.quantity), 0)` mengubah NULL hasil SUM atas baris kosong menjadi 0 secara eksplisit.

Catatan: tidak ada penggunaan `EXISTS/IN (SELECT ...)/ANY/ALL` di codebase saat ini — kebutuhan query IOMS selama ini cukup terselesaikan dengan `LEFT JOIN/subquery` skalar. Ini gap yang diakui, bukan diklaim sudah diterapkan.

07 Transactions, Isolation, dan Failure Handling

Diterapkan

Operasi multi-tabel dibungkus transaksi eksplisit, dengan row lock untuk mencegah race condition — didokumentasikan di ADR-002.

app/Service/SalesOrderService.php :202, 231, 233

`beginTransaction()` sebelum proses fulfill, `commit()` setelah decrement stok + tulis ledger sukses, `rollback()` di blok catch bila stok tidak cukup.

app/Repository/MySQLProductStockRepository.php :53–64

`lockForUpdate()` — `SELECT quantity ... FOR UPDATE` mengunci baris sebelum dibaca, mencegah dua transaksi membaca nilai stok "available" yang sama secara bersamaan.

app/Service/PurchaseOrderService.php :184, 233, 235

Pola transaksi yang sama diterapkan pada goods receipt (increment stok + tulis ledger tipe Receipt).

tests/Integration/GoodsIssueIntegrationTest.php :37–62

Bukti test: goods issue kedua yang memperebutkan stok yang sama ditolak (`InsufficientStockException`) setelah goods issue pertama menghabiskan stok — terhadap MySQL nyata di Docker.

08 Views, Layered Views, dan Stored Procedures

Belum diterapkan

Tidak ada `CREATE VIEW` atau `CREATE PROCEDURE` di `database/schema.sql`. Seluruh agregasi dan business logic sengaja ditaruh di layer Service (PHP), bukan di database, mengikuti keputusan arsitektur ARCH-01 (lihat `docs/architecture/adr-001-repository-pattern.md`) agar logic tetap testable lewat unit test tanpa koneksi database nyata.

Trade-off: pendekatan ini konsisten dengan constraint brief project (backend PHP native, business rule di Service) tapi berarti tidak ada bukti penerapan konsep view/stored procedure dari modul ini.

09 Triggers: Power, Risk, dan Alternatives

Belum diterapkan — by design

Tidak ada trigger database. Perubahan `product_stocks` dan penulisan `stock_ledger` dilakukan eksplisit dari kode aplikasi (Service layer) di dalam satu transaksi PDO, bukan efek samping trigger yang tersembunyi dari pembaca kode.

[docs/architecture/adr-002-concurrency-safe-stock.md](#) :12–14

ADR membahas alternatif mekanisme locking, konsisten dengan prinsip Bab 9: trigger dihindari karena efek samping implisit membuat alur sulit ditelusuri dan diuji — keputusan menaruh logic di Service (eksplisit, testable) sejalan dengan risiko trigger yang dibahas modul.

10 OLTP/OLAP, Access Control, Log, dan Profiling

Sebagian

Least-privilege access control dan error logging sudah diterapkan; profiling query formal (slow query log/EXPLAIN ANALYZE) belum ada.

[docker-compose.yml](#) :26–29

Aplikasi konek ke MySQL sebagai `ioms_user` (bukan `root`) — least privilege: user aplikasi tidak punya hak admin penuh terhadap instance database.

[app/Controller/Controller.php](#) :75, 85, 96

Exception (authorization, domain, unhandled) dicatat lewat `error_log()` dengan konteks (pesan + file:line asal), tapi detail teknis tidak pernah diteruskan ke response user (lihat `public/index.php:31` — `display_errors=0`).

Catatan: tidak ada pemisahan user OLTP vs OLAP maupun query profiling terstruktur (mis. `slow_query_log`) — di luar skala kebutuhan IOMS saat ini.

11 Execution Plans dan Evidence-Based Tuning

Belum diterapkan

Tidak ada dokumen EXPLAIN/EXPLAIN ANALYZE before-after yang disimpan sebagai bukti tuning. Desain index (Bab 2) dilakukan berdasarkan pola akses yang diprediksi dari kode (kolom yang dipakai di WHERE/ORDER BY), bukan hasil pengukuran plan aktual.

Ini adalah gap paling jelas terhadap modul — kandidat evidence tambahan sebelum presentasi jika waktu memungkinkan: jalankan EXPLAIN pada query list PO/SO dan query low-stock, simpan hasilnya di `docs/quality/`.

12 N+1 Refactoring

Diterapkan

Dashboard awalnya melakukan 8 query COUNT terpisah (satu per status) untuk satu halaman — pola serupa N+1 — kemudian dikonsolidasi jadi 2 query GROUP BY.

app/Service/DashboardService.php :34–35

`$soCounts = $this->salesOrders->countsByStatus();` dan `$poCounts = $this->purchaseOrders->countsByStatus();` — menggantikan 8 pemanggilan `countByStatus()/countSearch()` per status.

app/Repository/MySqlSalesOrderRepository.php :177–184 ·

app/Repository/MySqlPurchaseOrderRepository.php :109–116

Implementasi `countsByStatus()`: satu query `SELECT status, COUNT(*) AS cnt ... GROUP BY status`, hasil dipetakan jadi `array<string,int>` di PHP.

docs/quality/refactor-log.md , entri #4

Didokumentasikan dengan format smell → teknik → before/after, termasuk hasil verifikasi angka dashboard dicocokkan manual terhadap GROUP BY langsung di MySQL.